

# THE **AI** CODEBASE ANALYST

**FROM ZERO TO A WORKING AGENT**

**LESSON 3**  
GIVING YOUR AGENT TOOLS

## Lesson 3: Giving Your Agent Tools

*"A computer is like a bicycle for our minds."*

— Steve Jobs

### The "Why": The Brain in a Jar

In the last lesson, we achieved a monumental milestone: we built our agent's brain. Using LangGraph, we created a state machine with memory, allowing it to hold a conversation. It can reason, it can remember, but it is fundamentally helpless.

Our agent is a **brain in a jar**.

It lives entirely within the confines of its conversation with the LLM. It has no ability to interact with the outside world. It has no hands to perform actions, no eyes to read data, no voice to speak to other systems. To have true *agency*, it must be able to *act*.

This lesson is about giving our agent its first set of hands. We will teach it how to use a **Tool**, a simple function that allows it to perform an action in the world. This is the moment our agent stops being a mere conversationalist and starts its journey to becoming a true, skilled intern.

### The "What": The ReAct Loop - Thought, Action, Observation

How does an agent decide to act? It follows a powerful mental model called **ReAct**, which stands for **Reason + Act**. This is the fundamental logic loop that drives all modern, autonomous agents. We will now build this loop explicitly into our LangGraph state machine.

The ReAct loop breaks down a complex task into a series of turns:

1. **Reason (Thought):** Based on the user's prompt and its memory, the LLM "thinks" about what to do next. It formulates a plan. *Example: "The user is asking for the word count. I don't know the answer myself, but I have a tool that can count words. I should use that tool."*

2. **Act (Action):** The agent translates that thought into a structured, executable command—a call to one of its available tools. *Example: tool\_call:*  
`count_words(text="...")`
3. **Observation:** The system executes the tool, and the tool returns a result. This result is the "Observation." *Example: tool\_output: 6*

This Thought -> Action -> Observation cycle repeats. The new Observation is fed back into the agent's memory, and it begins the next loop of reasoning until it has a final answer for the user.

To build this loop, we will upgrade our graph with two new components:

- A **ToolNode**: A specialized "worker" in our graph that knows how to execute tools.
- A **Conditional Edge**: A "router" or "traffic cop" that decides, after each LLM call, whether the agent should use a tool or provide its final answer.

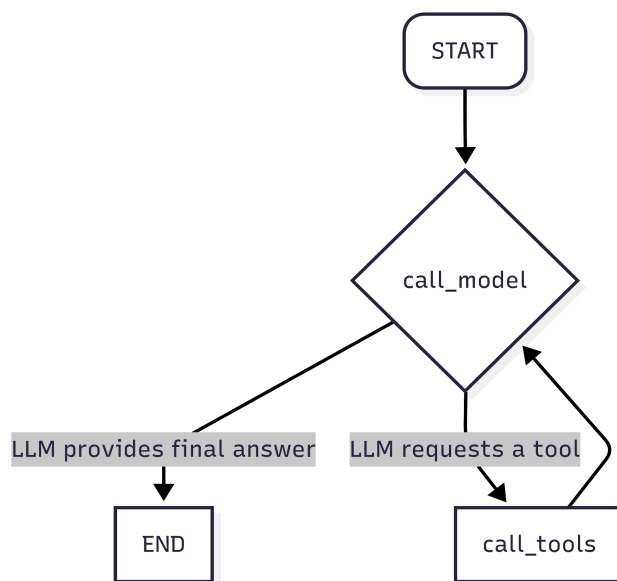


Figure 3-1: The architecture of our ReAct loop agent. The conditional edge after `call_model` allows the agent to decide whether to use a tool or finish.

## The "How" (The Build): Step-by-Step

Let's give our agent its first, simple skill: the ability to count words.

### 1. Activate Your Virtual Environment & Update Dependencies

Ensure your (venv) is active. We need to add a library that helps LangChain better integrate with our tools.

Update your requirements.txt file to include langchain\_community:

```
python-dotenv
langchain
langchain-openai
langsmith
langgraph
langchain_community
```

Then, run the installer:

```
pip install -r requirements.txt
```

### 2. Create Your First Tool

A tool is just a Python function decorated with `@tool` and a very clear docstring. The docstring is critical—it's the API documentation for the AI.

First, create a new folder for our tools:

```
mkdir tools
```

Inside the tools/ folder, create a new file named `__init__.py` (this makes it a Python package) and another file named `word_counter.py`.

- **File:** `tools/word_counter.py`

```
# tools/word_counter.py

from langchain_core.tools import tool

@tool
def count_words(text: str) -> int:
    """
    Counts the number of words in a given text.
    Use this when a user asks for the length of a document, a sentence,
    or any piece of text.
    The input should be the text whose words you want to count.
    """
    print(f"\n---EXECUTING WORD COUNT TOOL---")
    return len(text.split())
```

### 3. Architect the ReAct Graph

Now, we'll create a new script to build our more intelligent, tool-using agent.

- **File:** `03_agent_with_tools.py`

#### Pristine Code Block

```
# 03_agent_with_tools.py

import os
from dotenv import load_dotenv
```

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage, ToolMessage
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
from typing import TypedDict, List

# Import our new tool
from tools.word_counter import count_words

# --- 1. Load Environment Variables ---
load_dotenv()

# --- 2. Define the Agent's State ---
# We are adding a new key, `tool_calls`, to store the LLM's decision to
# use a tool.
class AgentState(TypedDict):
    messages: List[HumanMessage | AIMessage | ToolMessage]
    tool_calls: List[dict] # LangGraph uses this to route to the
    ToolNode

# --- 3. Define the Nodes ---
# The LLM "brain" of our agent. It will decide whether to respond or use
# a tool.
def call_model(state: AgentState):
    """A node that calls the LLM to generate a response or a tool
    call."""
    print("---CALLING THE MODEL---")
    llm = ChatOpenAI(
        base_url="https://openrouter.ai/api/v1",
        api_key=os.getenv("OPENROUTER_API_KEY"),
        model="google/gemini-2.5-flash",
```

```
)

# We bind our tool to the LLM. This makes the LLM "aware" of the tool.
llm_with_tools = llm.bind_tools([count_words])

response = llm_with_tools.invoke(state["messages"])

# LangGraph will automatically route to the ToolNode if `tool_calls` is present.
return {"messages": [response], "tool_calls": response.tool_calls}

# The node that executes the tools our agent decides to use.
tool_node = ToolNode([count_words])

# --- 4. Define the Edges (The Logic Flow) ---
def should_continue(state: AgentState):
    """
    The "Router". This function decides whether to continue using tools
    or to end the process and respond to the user.
    """
    print("---ASSESSING RESPONSE---")
    if state["tool_calls"]:
        # The LLM has requested to use a tool.
        return "continue"
    else:
        # The LLM has provided a final answer.
        return "end"

# --- 5. Build the Graph ---
graph_builder = StateGraph(AgentState)
```

```
# Add the nodes
graph_builder.add_node("call_model", call_model)
graph_builder.add_node("call_tools", tool_node)

# Set the entry point
graph_builder.set_entry_point("call_model")

# Add the conditional edge - this is the core of the ReAct loop
graph_builder.add_conditional_edges(
    "call_model",
    should_continue,
    {
        "continue": "call_tools", # If tools are called, go to the tool
node
        "end": END                # If no tools, end the graph
    }
)

# Add the normal edge to loop back from the tool node to the model
graph_builder.add_edge("call_tools", "call_model")

# --- 6. Compile and Run the Agent ---
agent_graph = graph_builder.compile()

if __name__ == "__main__":
    query = "How many words are in the text 'I am a skilled AI
engineer'?"

    # The input to the graph is the initial state.
    initial_state = {
```

```
"messages": [HumanMessage(content=query)],
"tool_calls": []
}

print(f"Human: {query}\n")

# .stream() Lets us see the output of each node as it executes.
for event in agent_graph.stream(initial_state):
    for key, value in event.items():
        print(f"--- Event: {key} ---")
        print(value)
        print("-----")

# .invoke() runs the graph to completion and gives the final state.
final_state = agent_graph.invoke(initial_state)
final_answer = final_state["messages"][-1].content

print("\n--- Agent's Final Answer ---")
print(final_answer)
```

## Deconstructing the Script

- **Section 2: AgentState Update:** We've added `tool_calls` to our state. This is a special key that LangGraph looks for to know when to route to a ToolNode.
- **Section 3: `llm.bind_tools()`:** This is a critical step. It modifies the LLM client, so that every time we call it, it is aware of the tools we've provided. It's like giving our intern a list of software they have installed.
- **Section 3: ToolNode:** This is a powerful, pre-built node from LangGraph. Its job is simple: it looks at the `tool_calls` in the state, executes the correct tool with the correct arguments, and adds the result back to the messages list as a ToolMessage.

- **Section 4: The should\_continue Edge:** This function is the **router** of our agent. After the call\_model node runs, this function inspects the state. If the LLM's last response contained a request to use a tool (state["tool\_calls"] is not empty), it returns the string "continue". Otherwise, it returns "end".
- **Section 5: Building the Graph:** We now have a loop!
  1. We start at call\_model.
  2. The add\_conditional\_edges router checks the output. If it's "continue", it goes to call\_tools. If it's "end", the process finishes.
  3. The final add\_edge creates the loop: after call\_tools runs, we always go back to call\_model to let the agent "think" about the tool's output.

---

## The Integrated Run: The ReAct Loop in Action

Run the script from your terminal:

```
python 03_agent_with_tools.py
```

### Expected Console Output:

You will now see the step-by-step execution of the ReAct loop printed to your console.

```
Human: How many words are in the text 'I am a skilled AI engineer'?
```

```
--- Event: call_model ---
```

```
{'messages': [AIMessage(content='', additional_kwargs={'tool_calls':  
[{'index': 0, 'id': '...', 'function': {'arguments': '{"text": "I am a  
skilled AI engineer"}', 'name': 'count_words'}, 'type': 'function'}]]],  
'tool_calls': [ParsedToolCall(id='...', func_name='count_words',  
func_args={'text': 'I am a skilled AI engineer'})]}
```

```
-----
```

```
---ASSESSING RESPONSE---
---EXECUTING WORD COUNT TOOL---

--- Event: call_tools ---
{'messages': [ToolMessage(content='6', tool_call_id='...')]}
-----
---CALLING THE MODEL---

--- Event: call_model ---
{'messages': [AIMessage(content='The text "I am a skilled AI engineer"
contains 6 words.')]}, 'tool_calls': []}
-----
---ASSESSING RESPONSE---
--- Agent's Final Answer ---
The text "I am a skilled AI engineer" contains 6 words.
```

## Analysis: The Glass Box View in LangSmith

This terminal output is great, but LangSmith gives us the true "glass box" view. Go to your project and open the latest trace. You will see a beautiful, visual representation of the ReAct loop you just built.

Let's deconstruct the trace:

1. **First call\_model:** Click on this step. In the output, you will see the tool\_calls. The LLM has made a *decision*. It didn't answer; it decided to act.
2. **call\_tools:** This step shows the execution of our word\_counter tool. The input is the text, and the output is a ToolMessage containing the number 6. This is the **Observation**.

3. **Second call\_model:** The agent now re-evaluates. The input to this step now includes the user's question, its own tool call, *and* the tool's output. With this complete context, it can now synthesize the final, human-readable answer.

You have just debugged your agent's entire thought process.

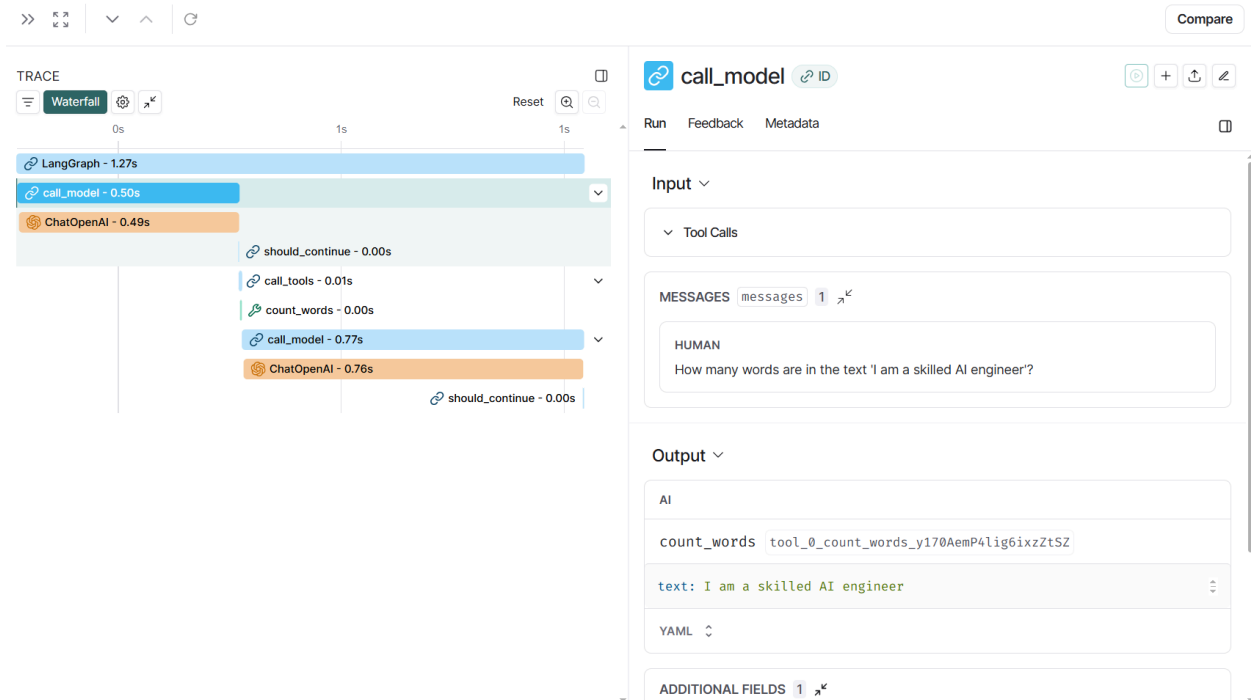


Figure 3-1: From Thinking to Acting. This LangSmith trace is the first visual proof of agency. The waterfall view on the left shows the full reasoning loop, while the detailed view on the right exposes the LLM's explicit decision: not to answer directly, but to call the count\_words tool to get the facts.

## Summary & Key Takeaways

This lesson was a monumental leap. Our agent is no longer just a brain in a jar. It has hands. It can act. You have engineered the fundamental loop of autonomous AI systems.

---

## Mental Models You've Gained:

- **The ReAct Loop:** You now deeply understand the Thought -> Action -> Observation cycle that powers modern agents.
- **Agent as a Router:** You see that a key part of an agent's job is not just answering questions, but routing requests to the correct tool.
- **Docstrings as AI Documentation:** You recognize that a tool's docstring is a critical piece of the prompt, directly influencing the agent's behavior.
- **Graphs for Control:** You have seen how LangGraph's explicit structure gives you perfect control and visibility over a complex, cyclical process.

## What's Next

Our agent has a simple skill, but to become the **AI Codebase Analyst**, it needs a superpower: the ability to read and understand an entire codebase. Its current memory is limited to the conversation.

In Lesson 4, we will begin the two-part process of giving our agent a true, long-term memory. We will dive into the theory and practice of **Retrieval-Augmented Generation (RAG)** and build the ingestion pipeline that transforms a folder of code into a searchable knowledge base.

---

*P.S. Proud of your first working agent? We'd love to see it! Share a screenshot of your agent's terminal output in the [#🚀-show-your-work](#) [channel on Discord](#).*